Week 1, Day 5: Executors, Locks, Deadlock and Sliding Window

Prepared for Upendra Kumar Sharma | 3–4 hours | Main topic: ExecutorService, thread pools, locks, deadlock, ThreadLocal | DSA: Sliding window

Today's Schedule

- **0:00–0:50** Study ExecutorService, ThreadPoolExecutor and pool sizing (sections 1 to 3).
- **0:50–1:30** Study ReentrantLock, deadlock and the synchronizers (sections 4 to 6).
- **1:30–1:45** Study ThreadLocal (section 7).
- **1:45–2:45** DSA: solve the 3 sliding window problems (section 8). Spend at most 25 minutes per problem before reading the hint.
- **2:45–3:15** Answer the interview questions aloud (section 9) and add 10 flashcards.

1. Why Executors

- Creating a raw thread per task is expensive and unbounded. Under load you can run out of memory or thrash the CPU.
- An **ExecutorService** separates task submission from execution. A pool reuses a fixed set of threads and queues extra work.
- **execute(Runnable)** returns nothing. **submit(Callable or Runnable)** returns a Future. Exceptions thrown inside a submitted task are stored in the Future and only surface when you call get(), so a forgotten Future silently hides failures.
- **Shutdown:** shutdown() stops accepting tasks and lets queued ones finish. shutdownNow() interrupts running tasks and returns the queued ones. Follow shutdown() with awaitTermination(timeout).

Factory methods in Executors

- **newFixedThreadPool(n):** n threads with an unbounded queue. Under sustained overload the queue grows until memory runs out.
- **newCachedThreadPool():** creates threads as needed with no upper limit. Under overload it can create thousands of threads.
- **newSingleThreadExecutor():** one thread, tasks run in order.

- **newScheduledThreadPool(n):** delayed and periodic tasks.
- **newVirtualThreadPerTaskExecutor() (Java 21):** one cheap virtual thread per task, suited to blocking I/O work.
- **Production advice:** create a ThreadPoolExecutor directly with a bounded queue, a named thread factory and an explicit rejection policy, rather than using the Executors factory methods.

2. ThreadPoolExecutor Behavior

Constructor parameters: corePoolSize, maximumPoolSize, keepAliveTime, workQueue, threadFactory, rejectedExecutionHandler.

How a new task is handled, in order

1. If fewer than corePoolSize threads are running, start a new thread for the task.
2. Otherwise put the task in the queue.
3. If the queue is full and fewer than maximumPoolSize threads exist, start an extra thread.
4. If the queue is full and the maximum is reached, apply the rejection policy.

Note that extra threads beyond core only appear **after the queue is full**. With an unbounded queue, maximumPoolSize is never used.

Rejection policies

- **AbortPolicy (default):** throws RejectedExecutionException.
- **CallerRunsPolicy:** the submitting thread runs the task itself, which slows producers down and gives natural back-pressure.
- **DiscardPolicy:** silently drops the new task.
- **DiscardOldestPolicy:** drops the oldest queued task and retries.

```
ThreadPoolExecutor pool = new ThreadPoolExecutor(
    4, 8, 60, TimeUnit.SECONDS,
    new ArrayBlockingQueue<>(100),
    new ThreadFactoryBuilder().setNameFormat("order-worker-%d").build(),
    new ThreadPoolExecutor.CallerRunsPolicy());
```

(ThreadFactoryBuilder comes from Guava. Without Guava, write a small ThreadFactory that names its threads.)

3. Sizing a Thread Pool

- **CPU-bound work** (computation): threads about equal to the number of cores, or cores + 1.
- **I/O-bound work** (database, HTTP calls): threads = cores x (1 + wait time / compute time). In practice you start with a reasonable number, measure, and adjust.
- Use separate pools for separate kinds of work, so a slow downstream service cannot starve unrelated tasks (the bulkhead idea).
- Name your threads, monitor queue size, active count and rejected tasks, and expose them as metrics.
- **ForkJoinPool.commonPool** is used by parallel streams and by CompletableFuture's async methods by default. It is sized for CPU work, so pass your own executor when tasks block on I/O.

4. Locks: ReentrantLock and Friends

- **ReentrantLock** is an explicit lock with more control than synchronized: tryLock() (with or without timeout), lockInterruptibly(), optional fairness, and multiple Condition objects.
- **Always unlock in a finally block**, otherwise an exception leaves the lock held forever.
- **ReadWriteLock (ReentrantReadWriteLock):** many readers at once or a single writer. Good when reads vastly outnumber writes.
- **StampedLock:** adds optimistic reads for even better read throughput, but is not reentrant.

```
private final ReentrantLock lock = new ReentrantLock();

public boolean transfer(Account from, Account to, long amount) {
    if (lock.tryLock()) {
        try {
            // critical section
            return true;
        } finally {
            lock.unlock();
        }
    }
    return false;
}
```

synchronized vs ReentrantLock

- synchronized is simpler, released automatically, and good enough for most code.
- ReentrantLock is better when you need timeouts, interruptible waits, fairness, or several wait conditions.

5. Deadlock, Livelock and Starvation

Deadlock: two or more threads wait for each other forever. It needs all four conditions (Coffman conditions):

1. **Mutual exclusion:** a resource can be held by only one thread.
2. **Hold and wait:** a thread holds one lock while waiting for another.
3. **No preemption:** a lock cannot be taken away from its holder.
4. **Circular wait:** a cycle of threads each waiting for a lock held by the next.

```
// Thread 1: locks A then B        Thread 2: locks B then A  -> deadlock
synchronized (a) {
    synchronized (b) { /* work */ }
}
```

Prevention

- **Always acquire locks in one global order** (for example by account id). This breaks circular wait and is the most common fix.
- Use tryLock with a timeout, and back off and retry if you cannot get both locks.
- Avoid nested locks, keep critical sections small, and never call unknown (alien) code while holding a lock.

Detection: take a thread dump with jstack or jcmd Thread.print. The JVM reports "Found one Java-level deadlock" and lists the threads and locks involved. Tools such as VisualVM also show it.

Livelock: threads keep reacting to each other and make no progress, for example two threads repeatedly backing off and retrying in lockstep. Adding random delay helps.

Starvation: a thread never gets the CPU or a lock because others always win. Fair locks and sensible priorities reduce it.

6. Synchronizers

- **CountDownLatch:** one or more threads wait until a count reaches zero. One-shot. Example: wait for N services to finish starting.
- **CyclicBarrier:** a group of threads wait for each other at a barrier, and it can be reused.
- **Semaphore:** limits concurrent access to a resource using permits. Example: cap simultaneous calls to a third-party API at 10.
- **Phaser and Exchanger** exist for advanced cases. Only mention them.

7. ThreadLocal

- Gives each thread its own independent copy of a variable. Typical uses: per-request context such as user id or trace id, and non-thread-safe helpers such as SimpleDateFormat.
- **The pool danger:** pool threads are reused and live for a long time. A value set during one request stays attached to the thread and can leak into the next request, and it also keeps the object from being garbage collected.
- **Rule:** call remove() in a finally block when the request finishes. Frameworks do this for you in filters, but your own ThreadLocals are your responsibility.
- Context does not flow automatically into child threads or async tasks. Pass it explicitly, use InheritableThreadLocal carefully, or use a task decorator that copies context.

```
private static final ThreadLocal<String> TRACE_ID = new ThreadLocal<>();

void handle(Request req) {
    TRACE_ID.set(req.traceId());
    try {
        process(req);
    } finally {
        TRACE_ID.remove();
    }
}
```

8. DSA: Sliding Window (3 Problems)

Problem 1: Maximum Average Subarray I

- **Idea:** fixed-size window of length k. Compute the sum of the first k elements, then slide: add the new element, subtract the one leaving.
- **Complexity:** O(n) time, O(1) space.
- **Edge cases:** all negative numbers (do not start the maximum at 0), k equal to the array length.

```
public double findMaxAverage(int[] nums, int k) {
    long sum = 0;
    for (int i = 0; i < k; i++) sum += nums[i];
    long best = sum;
    for (int i = k; i < nums.length; i++) {
        sum += nums[i] - nums[i - k];
        best = Math.max(best, sum);
    }
    return (double) best / k;
}
```

Problem 2: Minimum Window Substring

- **Idea:** variable window. Count the characters needed from t. Expand the right edge, decreasing the needed count and tracking how many required characters are still missing. When nothing is missing, record the window and shrink from the left while it stays valid.
- **Complexity:** O(n + m) time, O(1) space for a fixed alphabet.
- **Edge cases:** t longer than s, repeated characters in t, no valid window (return an empty string).

```
public String minWindow(String s, String t) {
    if (t.length() > s.length()) return "";
    int[] need = new int[128];
    for (char c : t.toCharArray()) need[c]++;
    int missing = t.length(), left = 0, bestStart = 0, bestLen =
Integer.MAX_VALUE;
    for (int right = 0; right < s.length(); right++) {
        if (need[s.charAt(right)]-- > 0) missing--;
        while (missing == 0) {
            if (right - left + 1 < bestLen) {
                bestLen = right - left + 1;
                bestStart = left;
            }
            if (++need[s.charAt(left)] > 0) missing++;
            left++;
        }
    }
    return bestLen == Integer.MAX_VALUE ? "" : s.substring(bestStart,
bestStart + bestLen);
}
```

Problem 3: Permutation in String

- **Idea:** fixed window of length s1.length() over s2. Keep a count array of s1 minus the window's letters. When all 26 counts are zero, the window is a permutation of s1.
- **Complexity:** O(n x 26), effectively O(n), and O(1) space.
- **Edge cases:** s1 longer than s2, identical strings, repeated letters.

```
public boolean checkInclusion(String s1, String s2) {
    int n1 = s1.length();
    if (n1 > s2.length()) return false;
    int[] cnt = new int[26];
    for (char c : s1.toCharArray()) cnt[c - 'a']++;
    for (int i = 0; i < s2.length(); i++) {
        cnt[s2.charAt(i) - 'a']--;
        if (i >= n1) cnt[s2.charAt(i - n1) - 'a']++;
        if (allZero(cnt)) return true;
    }
    return false;
}
private boolean allZero(int[] a) {
    for (int x : a) if (x != 0) return false;
    return true;
}
```

9. Interview Questions to Answer Aloud

1. Why use an ExecutorService instead of creating threads yourself?
2. Explain how ThreadPoolExecutor handles a new task (core, queue, max, reject).
3. Why are newFixedThreadPool and newCachedThreadPool risky in production?
4. How do you size a thread pool for CPU-bound vs I/O-bound work?
5. Which rejection policies exist, and when would you use CallerRunsPolicy?
6. What is the difference between submit() and execute()? What happens to exceptions in each?
7. shutdown() vs shutdownNow(). How do you shut a pool down gracefully?
8. synchronized vs ReentrantLock: when would you pick the lock?
9. What are the four conditions for deadlock, and how do you prevent it?
10. How would you detect a deadlock in production?
11. What is the difference between deadlock, livelock and starvation?
12. CountDownLatch vs CyclicBarrier vs Semaphore, with a use case for each.
13. Why does ThreadLocal cause leaks in thread pools, and how do you avoid it?

10. End-of-Day Checklist

- [ ] I can explain the ThreadPoolExecutor task flow and the four rejection policies.
- [ ] I can size a pool for CPU-bound and I/O-bound work and justify the numbers.
- [ ] I can write a deadlock example and fix it with consistent lock ordering.
- [ ] I can name the four deadlock conditions and how to read a thread dump.
- [ ] I solved all 3 sliding window problems and can state their complexities.
- [ ] I wrote at least 10 flashcards from questions I got wrong.

Tomorrow (Day 6, weekend): CompletableFuture, Streams edge cases, Java 8 to 17 features, and a self-mock on Java core.